

프로그래밍 역량 강화 전문기관, 민코딩

Restaurant Booking



목차

1. Restaurant Booking KATA 소개
2. 1 단계 : 테스트 케이스 작성 (with Mocking)
3. 2 단계 : 테스트 케이스 리팩토링 (with Mocking Library)

Legacy Code Refactoring

Restaurant Booking KATA 소개

레스토랑

✓레스토랑에서 예약 완료 처리 해주는 프로그램

1. 고객의 예약을 받는다. (사람수, 시간)
2. 예약 스케줄에 추가한다.
3. 예약에 문제가 생기면 Exception을 발생시킨다.
4. 예약에 아무런 문제가 없으면, 고객에게 SMS를 보낸다.
5. 고객이 email도 가지고 있다면, 고객에게 email을 또 보낸다.



레스토랑 규칙

✓예약 규칙 (규칙을 어기면, Exception 발생)

1. 정시에만 예약을 받는다.
 - 09:03 예약이 들어오면 예외발생 (09:00은 가능)
2. 시간당 최대 수용인원이 정해져 있다.
 - 특정 시간에, 수용인원보다 더 많은 사람이 예약이 들어오면 예외 발생
3. 일요일에는 예약을 받지 않는다.



예약자 규칙

✓예약자 정보

- 이름, 전화번호는 필수이다.
- email은 옵션이다. (email을 주면, 예약 결과를 email로 받을 수 있다.)

✓스케줄 정보

- 예약자명
- 예약하고 싶은 날짜/시간
- 예약 인원 수



프로젝트 목적

✓레거시 코드를 리팩토링 하는 목적이 아니다.

- Mocking 과 함께 유닛테스트를 작성하고,
유닛테스트를 리팩토링 한다.

✓프로젝트 목적

1. 유닛테스트를 작성한다. 필요하면 Test Double을 직접 제작한다.
2. 작성한 유닛테스트를 Mocking Library를 사용하는 것으로 리팩토링 한다.

소스코드 준비

링크

- <https://github.com/mincoding1/RestaurantBooking>

소스코드 분석 1

예약자 정보 : 이름, 전화번호, 이메일

스케줄 정보 : 예약자 시간, 예약 인원수, **예약자 정보**

Customer.java

```
public class Customer {  
  
    private String name;  
    private String phoneNumber;  
    private String email;  
  
    public Customer(String name,String phoneNumber) {  
        this.name = name;  
        this.phoneNumber = phoneNumber;  
    }  
  
    public Customer(String name,String phoneNumber,String email) {  
        this.name = name;  
        this.phoneNumber = phoneNumber;  
        this.email = email;  
    }  
  
    public String getEmail() { return email; }  
}
```

Schedule.java

```
import java.time.LocalDateTime;  
  
public class Schedule {  
    private LocalDateTime dateTime;  
    private int numberOfPeople;  
    private Customer customer;  
  
    public Schedule(LocalDateTime dateTime, int numberOfPeople, Customer customer) {  
        this.dateTime = dateTime;  
        this.numberOfPeople = numberOfPeople;  
        this.customer = customer;  
    }  
  
    public LocalDateTime getDateTime() {  
        return dateTime;  
    }  
  
    public int getNumberOfPeople() { return numberOfPeople; }  
  
    public Customer getCustomer() { return customer; }  
}
```

소스코드 분석 2

- ✓ SMS / email 전송할 수 있는 객체
 - 구현이 완료되어 있다고 가정한다

SmsSender.java

```
public class SmsSender {  
  
    public void send(Schedule schedule) {  
        // send message to PhoneNumber  
    }  
  
}
```

MailSender.java

```
public class MailSender {  
  
    public void sendMail(Schedule schedule) {  
        // send mail  
    }  
  
}
```

소스코드 분석 3

✓예약을 받는 코드

- addSchedule() 메서드

BookingScheduler.java

```
public class BookingScheduler {
    private int capacityPerHour;
    private List<Schedule> schedules;
    private SmsSender smsSender;
    private MailSender mailSender;

    public BookingScheduler(int capacityPerHour) {
        this.schedules = new ArrayList<Schedule>();
        this.capacityPerHour = capacityPerHour;
        this.smsSender = new SmsSender();
        this.mailSender = new MailSender();
    }

    public void addSchedule(Schedule schedule) {

        // 정각에 예약하지 않을 경우 RuntimeException 발생
        if(schedule.getDateTime().getMinute() != 0 ){
            throw new RuntimeException("Booking should be on the hour.");
        }

        // 시간당 예약인원을 초과할 경우 RuntimeException 발생
        int numberOfPeople = schedule.getNumberOfPeople();
        for ( Schedule bookedSchedule : schedules ) {
            if ( bookedSchedule.getDateTime().isEqual(schedule.getDateTime()) ) {
                numberOfPeople += bookedSchedule.getNumberOfPeople();
            }
        }
        if (numberOfPeople > capacityPerHour){
            throw new RuntimeException("Number of people is over restaurant capacity per hour");
        }

        /*
        // 일요일에는 시스템을 오픈하지 않는다.
        LocalDateTime now = LocalDateTime.now();
        if(now.getDayOfWeek() == DayOfWeek.SUNDAY){
            throw new RuntimeException("Booking system is not available on sunday");
        }
        */

        schedules.add(schedule);

        // 고객에게 SMS 발송
        smsSender.send(schedule);
        // 고객이 E Mail을 가지고 있을 경우 E Mail 발송
        if(schedule.getCustomer().getEmail() != null){
            mailSender.sendMail(schedule);
        }
    }

    public boolean hasSchedule(Schedule schedule) {
        return schedules.contains(schedule);
    }

    public void setSmsSender(SmsSender smsSender) {
        this.smsSender = smsSender;
    }

    public void setMailSender(MailSender mailSender) {
        this.mailSender = mailSender;
    }
}
```

Unit Test

✓한글로 되어있음 (Test Case를 한글로 작성을 허용하는 팀도 존재)

- 코드 커버리지 설정을 위해, 테스트코드 Run을 1회 눌러주세요.
- 만약 Run이 안된다면, 다시 프로젝트를 Clone 받고 Run을 눌러주세요.

```
class BookingSchedulerTest {  
  
    @Test  
    public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {  
    }  
  
    @Test  
    public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {  
    }  
  
    @Test  
    public void 시간대별_인원제한이_있다_같은_시간대에_Capacity_초과할_경우_예외발생() {  
    }  
  
    @Test  
    public void 시간대별_인원제한이_있다_같은_시간대가_다르면_Capacity_차있어도_스케줄_추가_성공() {  
    }  
  
    @Test  
    public void 예약완료시_SMS는_무조건_발송() {  
    }  
  
    @Test  
    public void 이메일이_없는_경우에는_이메일_미발송() {  
    }  
  
    @Test  
    public void 이메일이_있는_경우에는_이메일_발송() {  
    }  
}
```

제출 준비

✓ Repository 이름

- RestaurantBooking-사번
- git push 하기

✓ Collaborator

- 팀원 & 강사

✓ Branch Protection Rule

- Approve 1 명 필수

✓ Branch명

- 1 단계 : feature/unittest
- 2 단계 : feature/mockito

필수 PR 횟수 : 2회

유닛 테스트를 작성한다. 필요시 Test Double을 직접 제작한다.

1 단계 : 테스트 케이스 작성
(with Mocking)

테스트 케이스 작성

✓Step1. "예약은 정시에만 가능하다." 테스트 작성 1

- 2021.3.26 의 **09:05**로 예약하여 Exception 발생 테스트
- import 등, 빌드 문제가 발생하면 Alt + Enter로 해결한다.

```
@Test
public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {
    //arrange
    DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
    LocalDateTime notOnTheHour = LocalDateTime.parse("2021/03/26 09:05", dateTimeFormatter);
    Customer customer = new Customer("Fake name", "010-1234-5678");
    Schedule schedule = new Schedule(notOnTheHour, 1, customer);
    BookingScheduler bookingScheduler = new BookingScheduler(3);

    //act
    assertThrows(RuntimeException.class, () -> {
        bookingScheduler.addSchedule(schedule);
    });

    //assert
    //expected runtime exception
}
```

테스트 케이스 작성

✓Step2. "예약은 정시에만 가능하다." 테스트 작성 2

- 2021.3.26 의 09:00로 예약하여 정상 예약 확인

```
@Test
public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {
    //arrange
    DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
    LocalDateTime onTheHour = LocalDateTime.parse("2021/03/26 09:00", dateTimeFormatter);
    Customer customer = new Customer("Fake name", "010-1234-5678");
    Schedule schedule = new Schedule(onTheHour, 1, customer);
    BookingScheduler bookingScheduler = new BookingScheduler(3);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(bookingScheduler.hasSchedule(schedule)).isEqualTo(true);
}
```


테스트 케이스 작성

✓Step3-1. 자주 사용할 코드를 final 필드로 옮기기

- Formatter, LocalDateTime, Customer를 필드로 직접 옮기기
- “public static final”을 붙여준다.

```
class BookingSchedulerTest {  
    @Test  
    public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {  
        //arrange  
        DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");  
        LocalDateTime notOnTheHour = LocalDateTime.parse("2021/03/26 09:05", dateTimeFormatter);  
        Customer customer = new Customer("Fake name", "010-1234-5678");  
        Schedule schedule = new Schedule(notOnTheHour, 1, customer);  
        BookingScheduler bookingScheduler = new BookingScheduler(3);  
  
        //act  
        assertThrows(RuntimeException.class, () -> {  
            bookingScheduler.addSchedule(schedule);  
        });  
  
        //assert  
        //expected runtime exception  
    }  
  
    @Test  
    public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {  
        //arrange  
        DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");  
        LocalDateTime onTheHour = LocalDateTime.parse("2021/03/26 09:00", dateTimeFormatter);  
        Customer customer = new Customer("Fake name", "010-1234-5678");  
        Schedule schedule = new Schedule(onTheHour, 1, customer);  
        BookingScheduler bookingScheduler = new BookingScheduler(3);  
    }  
}
```



```
class BookingSchedulerTest {  
    public static final DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");  
    public static final LocalDateTime onTheHour = LocalDateTime.parse("2021/03/26 09:00", dateTimeFormatter);  
    public static final LocalDateTime notOnTheHour = LocalDateTime.parse("2021/03/26 09:05", dateTimeFormatter);  
    public static final Customer customer = new Customer("Fake name", "010-1234-5678");  
  
    @Test  
    public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {  
        //arrange  
        Schedule schedule = new Schedule(notOnTheHour, 1, customer);  
        BookingScheduler bookingScheduler = new BookingScheduler(3);  
  
        //act  
        assertThrows(RuntimeException.class, () -> {  
            bookingScheduler.addSchedule(schedule);  
        });  
  
        //assert  
        //expected runtime exception  
    }  
  
    @Test  
    public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {  
        //arrange  
        Schedule schedule = new Schedule(onTheHour, 1, customer);  
        BookingScheduler bookingScheduler = new BookingScheduler(3);  
    }  
}
```

테스트 케이스 작성

✓Step3-2. final 필드 이름 변경

- dateTimeFormatter → FORMAT
- onTheHour → ON_THE_HOUR
- notOnTheHour → NOT_ON_THE_HOUR
- customer → CUSTOMER

```
public static final DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");  
public static final LocalDateTime onTheHour = LocalDateTime.parse("2021/03/26 09:00", dateTimeFormatter);  
public static final LocalDateTime notOnTheHour = LocalDateTime.parse("2021/03/26 09:05", dateTimeFormatter);  
public static final Customer customer = new Customer("Fake name", "010-1234-5678");
```



```
public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");  
public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:00", FORMAT);  
public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:05", FORMAT);  
public static final Customer CUSTOMER = new Customer("Fake name", "010-1234-5678");
```

테스트 케이스 작성

✓Step3-3. Magic number 치환

- 숫자 1 (numberOfPeople)을 UNDER_CAPACITY 로 변경 (상수 도입하기 리팩토링)
- 숫자 3 (capacityPerHour)을 CAPACITY_PER_HOUR로 변경 (상수 도입하기 리팩토링)

```
@Test
public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {
    //arrange
    Schedule schedule = new Schedule(NOT_ON_THE_HOUR, 1, CUSTOMER);
    BookingScheduler bookingScheduler = new BookingScheduler(3);

    //act
    assertThrows(RuntimeException.class, () -> {
        bookingScheduler.addSchedule(schedule);
    });

    //assert
    //expected runtime exception
}

@Test
public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, 1, CUSTOMER);
    BookingScheduler bookingScheduler = new BookingScheduler(3);
```



```
@Test
public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {
    //arrange
    Schedule schedule = new Schedule(NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    BookingScheduler bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);

    //act
    assertThrows(RuntimeException.class, () -> {
        bookingScheduler.addSchedule(schedule);
    });

    //assert
    //expected runtime exception
}

@Test
public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    BookingScheduler bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);
```

테스트 케이스 작성

✓Step3-4. 필드 & 생성자에서 초기화

- bookingScheduler를 public 필드로 추가
- 생성자를 추가하여, 초기화하기

```
@Test
public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {
    //arrange
    Schedule schedule = new Schedule(NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    BookingScheduler bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);

    //act
    assertThrows(RuntimeException.class, () -> {
        bookingScheduler.addSchedule(schedule);
    });

    //assert
    //expected runtime exception
}

@Test
public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    BookingScheduler bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);
```



```
public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:00", FORMAT);
public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:05", FORMAT);
public static final Customer CUSTOMER = new Customer("Fake name", "010-1234-5678");
public static final int UNDER_CAPACITY = 1;
private static final int CAPACITY_PER_HOUR = 3;

BookingScheduler bookingScheduler;

public BookingSchedulerTest() {
    bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);
}

@Test
public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {
    //arrange
    Schedule schedule = new Schedule(NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);

    //act
    assertThrows(RuntimeException.class, () -> {
        bookingScheduler.addSchedule(schedule);
    });

    //assert
    //expected runtime exception
}
```

테스트 케이스 작성

✓Step3-5. 결과 확인

- 지금까지 변경한 결과가
우측과 동일한지 확인한다.
- 시간당 최대 3인 제한으로
세팅한 것을 기억하자.

```
class BookingSchedulerTest {  
    public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");  
    public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:00", FORMAT);  
    public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:05", FORMAT);  
    public static final Customer CUSTOMER = new Customer("Fake name", "010-1234-5678");  
    public static final int UNDER_CAPACITY = 1;  
    private static final int CAPACITY_PER_HOUR = 3;  
  
    BookingScheduler bookingScheduler;  
  
    public BookingSchedulerTest() {  
        bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);  
    }  
  
    @Test  
    public void 예약은_정시에만_가능하다_정시가_아닌경우_예약불가() {  
        //arrange  
        Schedule schedule = new Schedule(NOT_ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);  
  
        //act  
        assertThrows(RuntimeException.class, () -> {  
            bookingScheduler.addSchedule(schedule);  
        });  
  
        //assert  
        //expected runtime exception  
    }  
  
    @Test  
    public void 예약은_정시에만_가능하다_정시인_경우_예약가능() {  
        //arrange  
        Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);  
  
        //act  
        bookingScheduler.addSchedule(schedule);  
  
        //assert  
        assertThat(bookingScheduler.hasSchedule(schedule)).isEqualTo(true);  
    }  
}
```

테스트 케이스 작성

✓Step4. 최대 수용 인원을 초과했을 때, 예외 발생 테스트

- 3인 스케줄을 추가한다.
- 동시간에 1인 스케줄 하나 더 추가했을 때 예외 발생 테스트

```
@Test
public void 시간대별_인원제한이_있다_같은_시간대에_Capacity_초과할_경우_예외발생() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER);
    bookingScheduler.addSchedule(schedule);

    //act
    try {
        Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
        bookingScheduler.addSchedule(newSchedule);
        fail();
    }
    catch (RuntimeException e){
        //assert
        assertThat(e.getMessage()).isEqualTo("Number of people is over restaurant capacity per hour");
    }
}
```

테스트 케이스 작성

✓Step5. 인원제한을 채우고, 다른 시간대에 예약 걸면 정상임을 확인하기

- 3인 스케줄을 추가한다.
- ON_THE_HOUR 시간 + 1 시간으로 예약했을 때 정상임을 확인한다.

```
@Test
public void 시간대별_인원제한이_있다_같은_시간대가_다르면_Capacity_차있어도_스케줄_추가_성공() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, CAPACITY_PER_HOUR, CUSTOMER);
    bookingScheduler.addSchedule(schedule);

    //act
    LocalDateTime differentHour = ON_THE_HOUR.plusHours(1);
    Schedule newSchedule = new Schedule(differentHour, UNDER_CAPACITY, CUSTOMER);
    bookingScheduler.addSchedule(newSchedule);

    //assert
    assertThat(bookingScheduler.hasSchedule(schedule)).isEqualTo(true);
}
```

테스트 케이스 작성

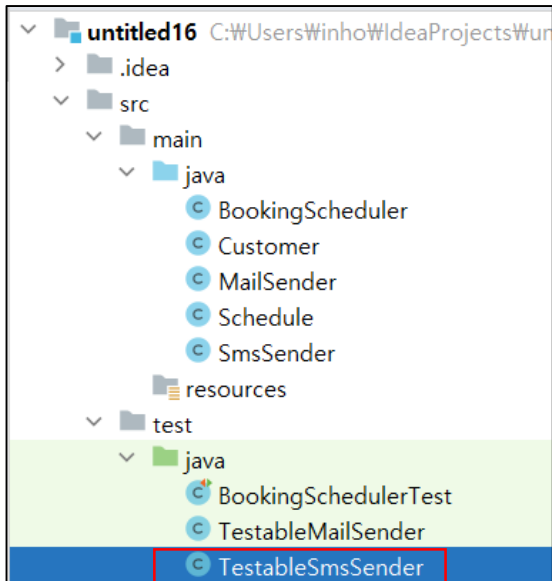
✓Step6. SMS 테스트

- 예약완료시 SMS가 전송되는지 Unit Test를 작성하고자 한다.
- 하지만, 테스트를 할 때마다 SMS를 보내게 되면 비용이 발생하게 된다.
 - **Test Double 필요!**
- Mocking Library를 사용하지 않고, SMS 대역(Double)을 직접 만들어보자.

테스트 케이스 작성

✓Step6-1. SMS Test Double 만들기

- 실제 객체(SmsSender)를 상속 받은 후, Overriding을 하여 가짜 객체를 생성한다.
- Behavior 검증을 위해, flag 변수를 하나 추가한다.



```
public class TestableSmsSender extends SmsSender {  
    private boolean sendMethodIsCalled;  
  
    @Override  
    public void send(Schedule schedule) {  
        System.out.println("테스트용 SmsSender class의 send메서드 실행됨");  
        sendMethodIsCalled = true;  
    }  
  
    public boolean isSendMethodIsCalled() {  
        return sendMethodIsCalled;  
    }  
}
```

TestableSmsSender.java

Mocking을 위해, 신규 Class 파일을 생성한다.

테스트 케이스 작성

✓Step6-2. Test Double로 Unit Test하기

- SmsSender 대신, 제작한 Test Double (TestableSmsSender) 클래스를 사용.
- 예약완료시, sender 메서드를 정상적으로 호출 하는지 Double로 테스트 가능

```
@Test
public void 예약완료시_SMS는_무조건_발송() {
    //arrange
    TestableSmsSender testableSmsSender = new TestableSmsSender();
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    bookingScheduler.setSmsSender(testableSmsSender);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(testableSmsSender.isSendMethodIsCalled()).isEqualTo(true);
}
```

테스트 케이스 작성

✓Step7-1. testableSmsSender 필드로 옮기기

- 지역변수에서 필드로 이동시키기

```
@Test
public void 예약완료시_SMS는_무조건_발송() {
    //arrange
    TestableSmsSender testableSmsSender = new TestableSmsSender();
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    bookingScheduler.setSmsSender(testableSmsSender);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(testableSmsSender.isSendMethodIsCalled()).isEqualTo(true);
}
```



```
class BookingSchedulerTest {
    public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPa
    public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("
    public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.par
    public static final Customer CUSTOMER = new Customer("Fake name", "01
    public static final int UNDER_CAPACITY = 1;
    private static final int CAPACITY_PER_HOUR = 3;

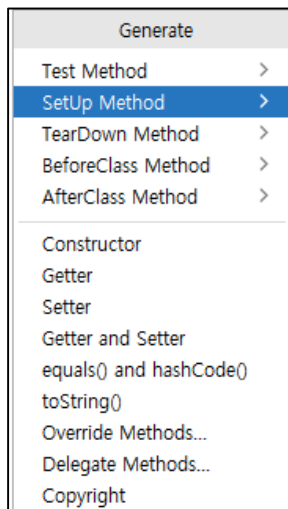
    BookingScheduler bookingScheduler;
    TestableSmsSender testableSmsSender = new TestableSmsSender();

    public BookingSchedulerTest() {
        bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);
    }
}
```

테스트 케이스 작성

✓Step7-2. SetUp 추가하기

- Generate (Alt + Insert) 메뉴에서 SetUp Method 선택하여, SetUp 메서드 생성하기
 - SetUp 메서드 : 각 테스트케이스 실행 직전, 수행되는 메서드



```
class BookingSchedulerTest {  
    public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");  
    public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:00", FORMAT);  
    public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:05", FORMAT);  
    public static final Customer CUSTOMER = new Customer("Fake name", "010-1234-5678");  
    public static final int UNDER_CAPACITY = 1;  
    private static final int CAPACITY_PER_HOUR = 3;  
  
    BookingScheduler bookingScheduler;  
    TestableSmsSender testableSmsSender = new TestableSmsSender();  
  
    public BookingSchedulerTest() {  
        bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);  
    }  
  
    @BeforeEach  
    void setUp() {  
  
    }  
}
```

테스트 케이스 작성

✓Step7-3. SmsSender 메서드 호출 코드 이동

- 기존 작성된 Test Double 세팅 코드를 setUp 메서드로 이동시키기.

```
@Test
public void 예약완료시_SMS는_무조건_발송() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    bookingScheduler.setSmsSender(testableSmsSender);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(testableSmsSender.isSendMethodIsCalled()).isEqualTo(true);
}
```



```
@BeforeEach
void setUp() {
    bookingScheduler.setSmsSender(testableSmsSender);
}
```

테스트 케이스 작성

✓Step7-4. 완성된 코드 확인하기

- 아래 코드와 다름이 없음을 확인하기

```
class BookingSchedulerTest {
    public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
    public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:00", FORMAT);
    public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:05", FORMAT);
    public static final Customer CUSTOMER = new Customer("Fake name", "010-1234-5678");
    public static final int UNDER_CAPACITY = 1;
    private static final int CAPACITY_PER_HOUR = 3;

    BookingScheduler bookingScheduler;
    TestableSmsSender testableSmsSender = new TestableSmsSender();

    public BookingSchedulerTest() {
        bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);
    }

    @BeforeEach
    void setUp() {
        bookingScheduler.setSmsSender(testableSmsSender);
    }
}
```

Unit Test 코드 시작부분

```
@Test
public void 예약완료시_SMS는_무조건_발송() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);

    //act
    bookingScheduler.addSchedule(schedule);

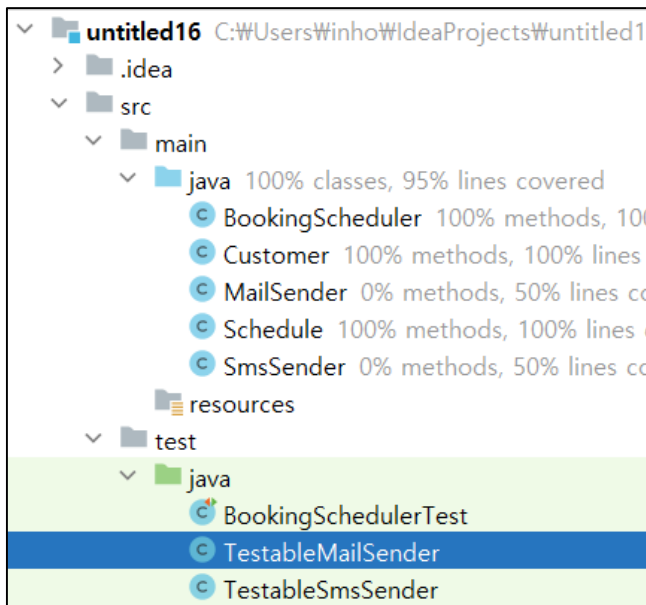
    //assert
    assertThat(testableSmsSender.isSendMethodIsCalled()).isEqualTo(true);
}
```

Step 7에서 작성한 테스트케이스

테스트 케이스 작성

✓Step8-1. Email Test

- 고객이 Email 주소가 존재한다면, Email을 보내야한다.
- Unit Test를 할 때마다 Email을 보낼 수 없기에 대역 (Test Double) 이 필요하다.
- Test Double용 새로운 Class를 생성한다.



TestableMailSender.java

```
public class TestableMailSender extends MailSender {  
    private int countSendMailMethodIsCalled;  
  
    @Override  
    public void sendMail(Schedule schedule) {  
        countSendMailMethodIsCalled++;  
    }  
  
    public int getCountSendMailMethodIsCalled() {  
        return countSendMailMethodIsCalled;  
    }  
}
```

테스트 케이스 작성

✓Step8-2. Email이 없을 때, 이메일 발송 여부 테스트

- 고객이 Email 주소가 없다면, Email을 보내면 안된다.
- 제작한 Test Double을 이용하여, send 메서드가 호출 되었는지 Behavior 검증을 한다.

```
@Test
public void 이메일이_없는_경우에는_이메일_미발송() {
    //arrange
    TestableMailSender testableMailSender = new TestableMailSender();
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);
    bookingScheduler.setMailSender(testableMailSender);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(testableMailSender.getCountSendMailMethodIsCalled()).isEqualTo(0);
}
```


테스트 케이스 작성

✓Step9. Email이 있을 때, 이메일 발송 여부 테스트

- 고객이 Email 주소가 있다면, Email을 보내야한다.
- 제작한 Test Double을 이용하여, send 메서드가 호출 되었는지 Behavior 검증을 한다.

```
@Test
public void 이메일이_있는_경우에는_이메일_발송() {
    //arrange
    Customer customerWithMail = new Customer("Fake Name", "010-1234-5678", "test@test.com");
    TestableMailSender testableMailSender = new TestableMailSender();
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, customerWithMail);
    bookingScheduler.setMailSender(testableMailSender);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(testableMailSender.getCountSendMailMethodIsCalled()).isEqualTo(1);
}
```

테스트 케이스 작성

✓Step10-1. 중복 코드 리팩토링

- Email Test Double을 사용한 두 가지 테스트케이스 리팩토링을 위해, 다음 필드를 준비한다.

```
class BookingSchedulerTest {
    public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
    public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:00", FORMAT);
    public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:05", FORMAT);
    public static final Customer CUSTOMER = new Customer("Fake name", "010-1234-5678");
    public static final Customer CUSTOMER_WITH_MAIL = new Customer("Fake Name", "010-1234-5678", "test@test.com");
    public static final int UNDER_CAPACITY = 1;
    private static final int CAPACITY_PER_HOUR = 3;

    BookingScheduler bookingScheduler;
    TestableSmsSender testableSmsSender = new TestableSmsSender();
    TestableMailSender testableMailSender = new TestableMailSender();

    public BookingSchedulerTest() {
        bookingScheduler = new BookingScheduler(CAPACITY_PER_HOUR);
    }

    @BeforeEach
    void setUp() {
        bookingScheduler.setSmsSender(testableSmsSender);
        bookingScheduler.setMailSender(testableMailSender);
    }
}
```

테스트 케이스 작성

✓Step10-2. 리팩토링 완료된, 테스트 코드

- Step 8, Step 9에서 작성된 테스트케이스

```
@Test
public void 이메일이_없는_경우에는_이메일_미발송() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(testableMailSender.getCountSendMailMethodIsCalled()).isEqualTo(0);
}

@Test
public void 이메일이_있는_경우에는_이메일_발송() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);

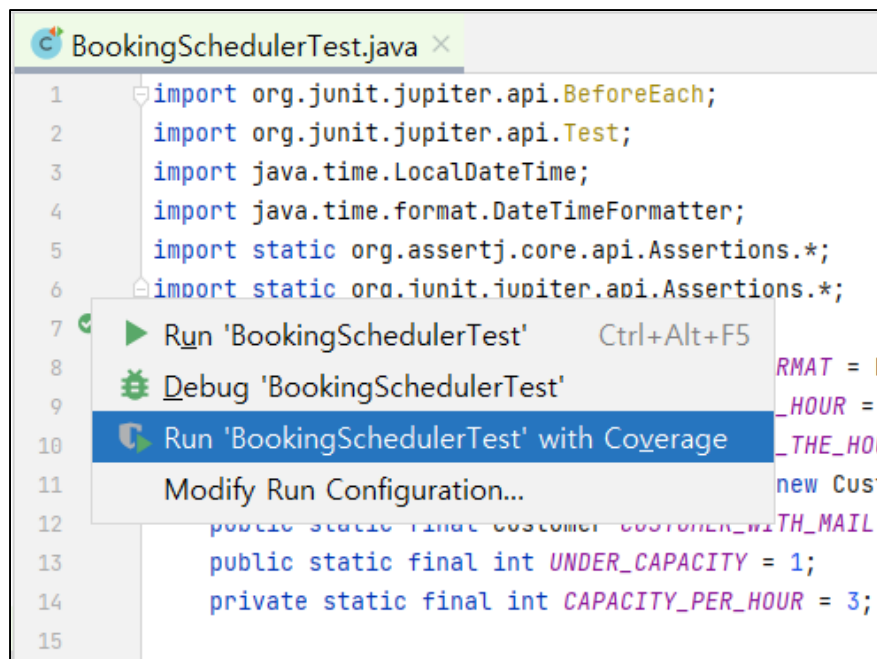
    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    assertThat(testableMailSender.getCountSendMailMethodIsCalled()).isEqualTo(1);
}
```

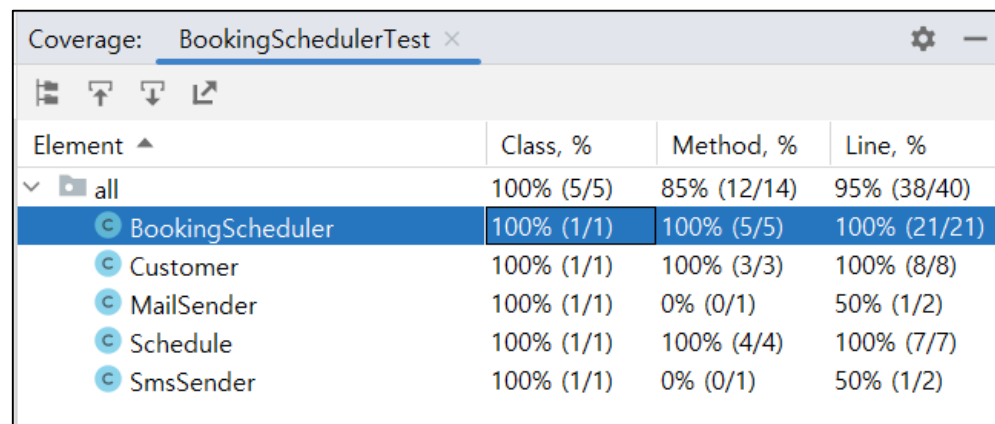
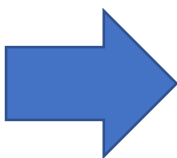
테스트 케이스 작성

✓Step11-1. 현재 Code Coverage 측정해보기

- BookingScheduler.java 기준, 100%가 나온다.



```
1 import org.junit.jupiter.api.BeforeEach;
2 import org.junit.jupiter.api.Test;
3 import java.time.LocalDateTime;
4 import java.time.format.DateTimeFormatter;
5 import static org.assertj.core.api.Assertions.*;
6 import static org.junit.jupiter.api.Assertions.*;
7
8 public class BookingSchedulerTest {
9     // ...
10     // ...
11     // ...
12     public static final Customer CUSTOMER_WITH_MAIL = ...;
13     public static final int UNDER_CAPACITY = 1;
14     private static final int CAPACITY_PER_HOUR = 3;
15 }
```



Element ^	Class, %	Method, %	Line, %
all	100% (5/5)	85% (12/14)	95% (38/40)
BookingScheduler	100% (1/1)	100% (5/5)	100% (21/21)
Customer	100% (1/1)	100% (3/3)	100% (8/8)
MailSender	100% (1/1)	0% (0/1)	50% (1/2)
Schedule	100% (1/1)	100% (4/4)	100% (7/7)
SmsSender	100% (1/1)	0% (0/1)	50% (1/2)

테스트 케이스 작성

✓Step11-2. 일요일 예약시 Exception 코드, 주석 풀기

- 주석을 푼 후, 다시 Code Coverage를 측정해본다. (95%로 줄어들었다.)
- Code Coverage를 다시 100%를 만들어야 한다.

```
public void addSchedule(Schedule schedule) {  
  
    // 정각에 예약하지 않을 경우 RuntimeException 발생  
    if(schedule.getDateTime().getMinute() != 0){  
        throw new RuntimeException("Booking should be on the hour.");  
    }  
  
    // 시간당 예약인원을 초과할 경우 RuntimeException 발생  
    int numberOfPeople = schedule.getNumberOfPeople();  
    for ( Schedule bookedSchedule : schedules ) {  
        if ( bookedSchedule.getDateTime().isEqual(schedule.getDateTime()) ) {  
            numberOfPeople += bookedSchedule.getNumberOfPeople();  
        }  
    }  
    if (numberOfPeople > capacityPerHour){  
        throw new RuntimeException("Number of people is over restaurant capacity per hour");  
    }  
  
    // 일요일에는 시스템을 오픈하지 않는다.  
    LocalDateTime now = LocalDateTime.now();  
    if(now.getDayOfWeek() == DayOfWeek.SUNDAY){  
        throw new RuntimeException("Booking system is not available on sunday");  
    }  
  
    schedules.add(schedule);  
  
    // 고객에게 SMS 발송  
    smsSender.send(schedule);  
    // 고객이 E Mail을 가지고 있을 경우 E Mail 발송  
    if(schedule.getCustomer().getEmail() != null){  
        mailSender.sendMail(schedule);  
    }  
}
```

주석풀기



Coverage: java in untitled16 ×			
Element ^	Class, %	Method, %	Line, %
all	100% (5/5)	85% (12/14)	93% (40/43)
BookingScheduler	100% (1/1)	100% (5/5)	95% (23/24)
Customer	100% (1/1)	100% (3/3)	100% (8/8)
MailSender	100% (1/1)	0% (0/1)	50% (1/2)
Schedule	100% (1/1)	100% (4/4)	100% (7/7)
SmsSender	100% (1/1)	0% (0/1)	50% (1/2)

테스트 케이스 작성

✓Step11-3. 대역 필요성, 이해하기

- 오늘 (PC 기준) 날짜의 정보를 가져온다.
- 오늘 날짜가 '일요일'이라면, Exception 처리한다.

✓Exception이 잘 되는지 테스트 하려면, 일요일에만 테스트가 가능하다.

항시 테스트 하기 위해서는? → 대역 필요 (Test Double)

- 무조건 일요일로 인식하는 대역(가짜) Class 필요

```
// 일요일에는 시스템을 오픈하지 않는다.  
LocalDateTime now = LocalDateTime.now();  
if(now.getDayOfWeek() == DayOfWeek.SUNDAY){  
    throw new RuntimeException("Booking system is not available on sunday");  
}
```

테스트 케이스 작성

✓Step11-4. Mocking을 위한 리팩토링

- LocalDateTime() 부분에 Extract Method를 한다.
- 대역 Class가 getNow 메서드를 Overriding 하여, 다른 날짜를 return하도록 처리예정

```
// 일요일에는 시스템을 오픈하지 않는다.  
LocalDateTime now = LocalDateTime.now();  
if(now.getDayOfWeek() == DayOfWeek.SUNDAY){  
    throw new RuntimeException("Booking system is not available on sunday");  
}  
  
schedules.add(schedule);  
  
// 고객에게 SMS 발송
```

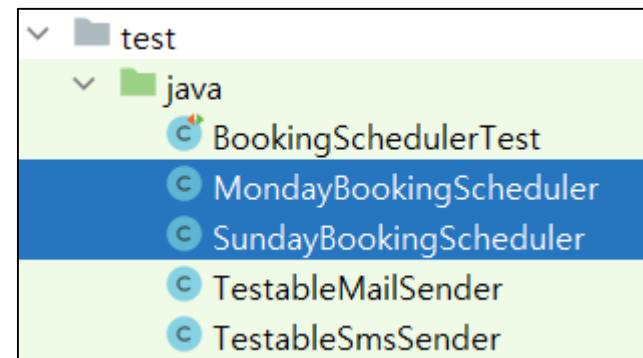
Refactor This		
1	Change Signature...	Ctrl+R, S
Extract/Introduce		
2	Introduce Variable...	Ctrl+R, V
3	Introduce Constant...	Ctrl+Alt+C
4	Introduce Field...	Ctrl+R, F
5	Introduce Parameter...	Ctrl+R, P
6	Introduce Functional Parameter...	Ctrl+Alt+Shift+P
7	Introduce Functional Variable...	
8	Extract Method...	Ctrl+R, M



```
// 일요일에는 시스템을 오픈하지 않는다.  
LocalDateTime now = getNow();  
if(now.getDayOfWeek() == DayOfWeek.SUNDAY){  
    throw new RuntimeException("Booking system is not available on sunday");  
}  
  
schedules.add(schedule);  
  
// 고객에게 SMS 발송  
smsSender.send(schedule);  
// 고객이 E Mail을 가지고 있을 경우 E Mail 발송  
if(schedule.getCustomer().getEmail() != null){  
    mailSender.sendMail(schedule);  
}  
}  
  
public LocalDateTime getNow() {  
    return LocalDateTime.now();  
}
```

public 으로 수정

테스트 케이스 작성



✓Step11-5. 대역 Class 2개 생성하기

■ BookingScheduler.java 의 대역 Class 2개 생성

- SundayBookingScheduler.java : getNow() 메서드는 무조건 일요일로 리턴한다.
- MondyBookingScheduler.java : getNow() 메서드는 무조건 월요일로 리턴한다.

SundayBookingScheduler.java

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class SundayBookingScheduler extends BookingScheduler{

    public SundayBookingScheduler(int capacityPerHour) {
        super(capacityPerHour);
    }

    @Override
    public LocalDateTime getNow() {
        DateTimeFormatter format = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
        return LocalDateTime.parse("2021/03/28 17:00", format);
    }
}
```

MondayBookingScheduler.java

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class MondayBookingScheduler extends BookingScheduler{

    public MondayBookingScheduler(int capacityPerHour) {
        super(capacityPerHour);
    }

    @Override
    public LocalDateTime getNow() {
        DateTimeFormatter format = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
        return LocalDateTime.parse("2024/06/03 17:00", format);
    }
}
```


테스트 케이스 작성

✓Step11-6. 일요일 Mocking을 사용하는 테스트 케이스 만들기

- 일요일 예약시, 정상적으로 Exception 발생이 되는지 테스트

```
@Test
public void 현재날짜가_일요일인_경우_예약불가_예외처리() {
    //arrange
    BookingScheduler bookingScheduler = new SundayBookingScheduler(CAPACITY_PER_HOUR);

    try {
        //act
        Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);
        bookingScheduler.addSchedule(newSchedule);
        fail();
    }
    catch (RuntimeException e){
        //assert
        assertThat(e.getMessage()).isEqualTo("Booking system is not available on sunday");
    }
}
```

테스트 케이스 작성

✓Step11-7. 월요일 Mocking을 사용하는 테스트 케이스 만들기

- 월요일 예약시, 정상적으로 예약이 되는지 확인하는 테스트 케이스
- 코드 커버리지가 100% 되는지 확인한다.

```
@Test
public void 현재날짜가_일요일이_아닌경우_예약가능() {
    //arrange
    BookingScheduler bookingScheduler = new MondayBookingScheduler(CAPACITY_PER_HOUR);

    //act
    Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);
    bookingScheduler.addSchedule(newSchedule);

    //assert
    assertThat(bookingScheduler.hasSchedule(newSchedule)).isEqualTo(true);
}
```

Coverage: java in untitled16

Element	Class, %	Method, %	Line, %
all	100% (5/5)	100% (15/15)	100% (44/44)
BookingScheduler	100% (1/1)	100% (6/6)	100% (25/25)
Customer	100% (1/1)	100% (3/3)	100% (8/8)
MailSender	100% (1/1)	100% (1/1)	100% (2/2)
Schedule	100% (1/1)	100% (4/4)	100% (7/7)
SmsSender	100% (1/1)	100% (1/1)	100% (2/2)

테스트 케이스 작성

✓Step12. 통합 Test Double 클래스 만들기

- SundayBookingScheduler, MondayBookingScheduler를 통합한 TestableScheduler 클래스 만들기

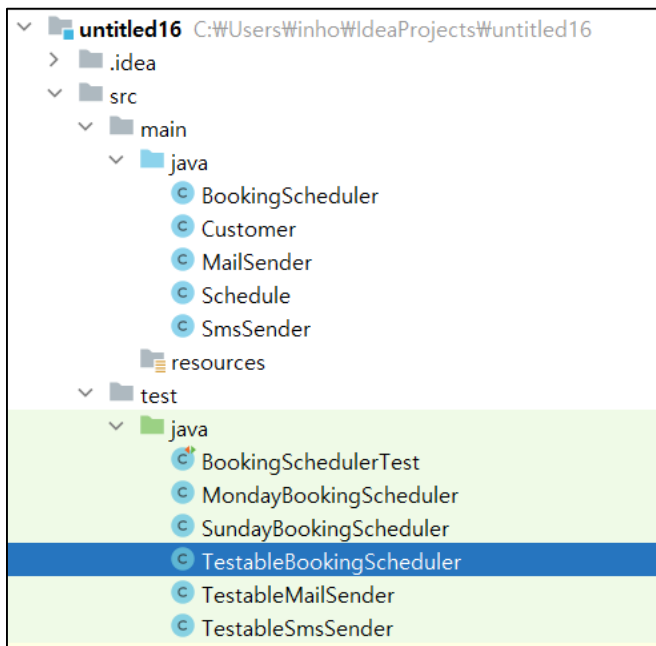
✓진행순서

1. TestableScheduler 클래스 만들기
2. 기존 테스트 코드를 TestableScheduler 클래스를 사용하도록 변경
3. SundayBookingScheduler, MondayBookingScheduler 클래스를 삭제

테스트 케이스 작성

✓Step12-1. TestableScheduler 클래스 만들기

- 원하는 날짜를 객체 생성시 지정할 수 있는 Test Double 클래스 제작



```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class TestableBookingScheduler extends BookingScheduler {
    private String dateTime;

    public TestableBookingScheduler(int capacityPerHour, String dateTime) {
        super(capacityPerHour);
        this.dateTime = dateTime;
    }

    @Override
    public LocalDateTime getNow() {
        DateTimeFormatter format = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
        return LocalDateTime.parse(dateTime, format);
    }
}
```

TestableBookingScheduler.java

[도전] 테스트 케이스 작성

✓Step12-2. 통합 Test Double 클래스로 대체하기 **Mission !**

- Step 11에서 만들었던 테스트케이스를 통합 Test Double 클래스로 대체하기
- 대체가 완료되었다면, 기존 SundayBookingScheduler, MondayBookingScheduler 클래스를 삭제한다.

```
@Test
public void 현재날짜가_일요일인_경우_예약불가_예외처리() {
    //arrange
    BookingScheduler bookingScheduler = new SundayBookingScheduler(CAPACITY_PER_HOUR);

    try {
        //act
        Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);
        bookingScheduler.addSchedule(newSchedule);
        fail();
    }
    catch (RuntimeException e){
        //assert
        assertEquals("Booking system is not available on sunday", e.getMessage());
    }
}

@Test
public void 현재날짜가_일요일이_아닌경우_예약가능() {
    //arrange
    BookingScheduler bookingScheduler = new MondayBookingScheduler(CAPACITY_PER_HOUR);

    //act
    Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);
    bookingScheduler.addSchedule(newSchedule);

    //assert
    assertEquals(bookingScheduler.hasSchedule(newSchedule), true);
}
```

Step 11의 두 테스트케이스를 리팩토링 한다.

Mockito를 사용한 유닛테스트 리팩토링

2 단계 : 테스트 케이스 리팩토링
(with Mocking Library)

Customer를 Mock으로 대체

✓Step 1-1 : 테스트 클래스에 Mockito 적용

- Mockito Library 를 설치한다.
- @ExtendWith 어노테이션 적용

BookingScheduler.java

```
1  import org.junit.jupiter.api.BeforeEach;
2  import org.junit.jupiter.api.Test;
3  import org.junit.jupiter.api.extension.ExtendWith;
4  import org.mockito.junit.jupiter.MockitoExtension;
5
6  import java.time.LocalDateTime;
7  import java.time.format.DateTimeFormatter;
8  import static org.assertj.core.api.Assertions.*;
9  import static org.junit.jupiter.api.Assertions.*;
10
11  @ExtendWith(MockitoExtension.class)
12  class BookingSchedulerTest {
13      public static final DateTimeFormatter FORMAT = DateTimeFormat
14      public static final LocalDateTime ON_THE_HOUR = LocalDateTime
15      public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime
```

Customer를 Mock으로 대체

✓Step 1-2 : Customer을 Mock 객체로 변경

- 실제 고객 정보를 입력할 필요가 없기에, **Mock 객체로 대체**
- answer 옵션
 - Default : 메서드의 return값은 항상 null
 - **RETURNS_MOCKS** : 적절한 return 값 수행

✓answer 옵션에 따른 메서드 리턴값

- CUSTOMER.getEmail() : **null** 리턴
- CUSTOMER_WITH_MAIL.getEmail() : **""** 리턴

```
@ExtendWith(MockitoExtension.class)
class BookingSchedulerTest {
    public static final DateTimeFormatter FORMAT = Da
    public static final LocalDateTime ON_THE_HOUR = L
    public static final LocalDateTime NOT_ON_THE_HOUR

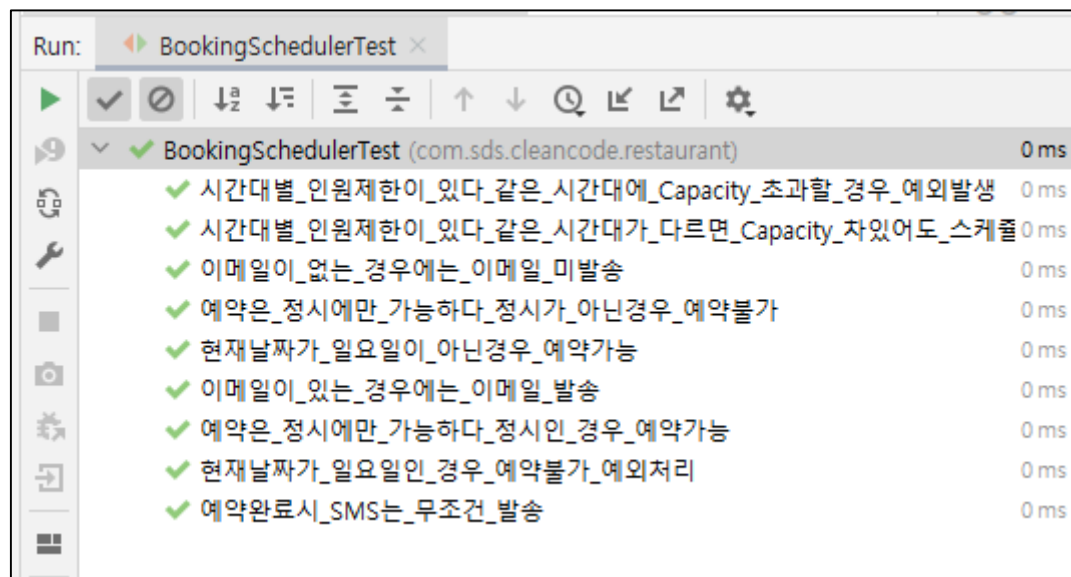
    @Mock
    public Customer CUSTOMER;

    @Mock(answer = Answers.RETURNS_MOCKS)
    public Customer CUSTOMER_WITH_MAIL;

    public static final int UNDER_CAPACITY = 1;
    private static final int CAPACITY_PER_HOUR = 3;
```


Customer를 Mock으로 대체

✓Step 1-3 : 결과 확인



TestableSender를 Mock 객체로 대체

✓ Step 2 : TestableSmsSender / TestableMailSender를 Mock 객체로 대체

- SmsSender, MailSender 객체 상속으로 만든, TestDouble
- 다음 페이지부터, 이 객체를 Mock 객체로 변경할 예정

```
TestableSmsSender testableSmsSender = new TestableSmsSender();
TestableMailSender testableMailSender = new TestableMailSender();

public BookingSchedulerTest() {
    bookingScheduler = new TestableBookingScheduler(CAPACITY_PER_HOUR, "2021/03/26 09:00");
}

@BeforeEach
void setUp() {
    bookingScheduler.setSmsSender(testableSmsSender);
    bookingScheduler.setMailSender(testableMailSender);
}
```

TestableSender를 Mock 객체로 대체

✓ Step 2-1 : TestableSmsSender / TestableMailSender를 Mock 객체로 대체

- 직접 만든 Test Double 객체 생성 대신, Mock 객체 생성으로 변경한다.

두 줄 제거

```
TestableSmsSender testableSmsSender = new TestableSmsSender();
TestableMailSender testableMailSender = new TestableMailSender();

public BookingSchedulerTest() {
    bookingScheduler = new TestableBookingScheduler(CAPACITY_PER_HOUR, "2021/03/26 09:00");
}

@BeforeEach
void setUp() {
    bookingScheduler.setSmsSender(testableSmsSender);
    bookingScheduler.setMailSender(testableMailSender);
}
```



Mock 객체
추가

```
@Mock
public SmsSender smsSender;

@Mock
public MailSender mailSender;

public BookingSchedulerTest() {
    bookingScheduler = new TestableBookingScheduler(CAPACITY_PER_HOUR, "2021/03/26 09:00");
}

@BeforeEach
void setUp() {
    bookingScheduler.setSmsSender(smsSender);
    bookingScheduler.setMailSender(mailSender);
}
```

TestableSender를 Mock 객체로 대체

✓Step 2-2 : verify로 Mock 객체 행위 검증 (smsSender)

- send 메서드 1회 호출되어야 Pass

```
@Test
public void 예약완료시_SMS는_무조건_발송() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    //assertThat(testableSmsSender.isSendMethodIsCalled()).isEqualTo(true);
    verify(smsSender, times(1)).send(schedule);
}
```

verify 에서 발생하는 import 문제는 Alt + Enter로 해결한다.

TestableSender를 Mock 객체로 대체

✓Step 2-3 : verify로 Mock 객체 행위 검증 (mailSender)

- send 메서드 0회 / 1회 호출되어야 Pass

```
@Test
public void 이메일이_없는_경우에는_이메일_미발송() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER);

    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    //assertThat(testableMailSender.getCountSendMailMethodIsCalled()).isEqualTo(0);
    verify(mailSender, times(0)).sendMail(schedule);
}

@Test
public void 이메일이_있는_경우에는_이메일_발송() {
    //arrange
    Schedule schedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);

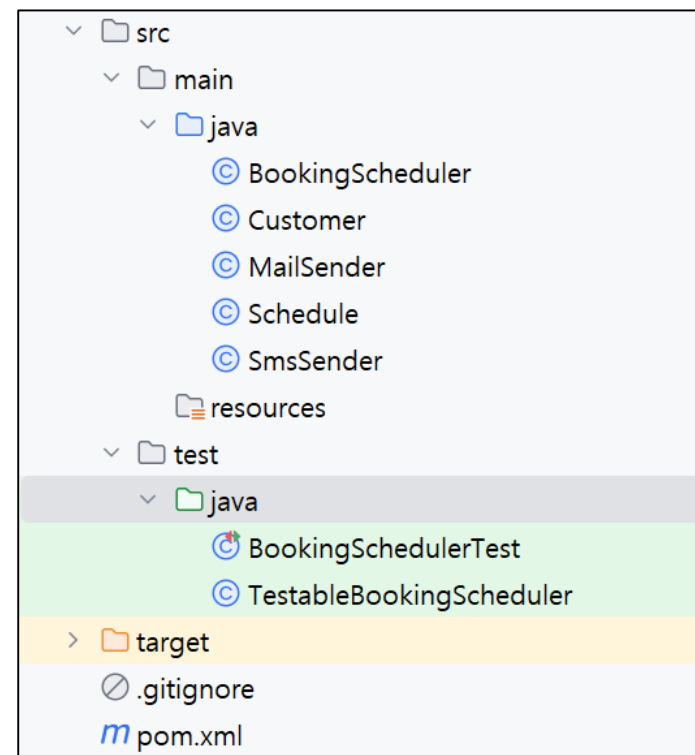
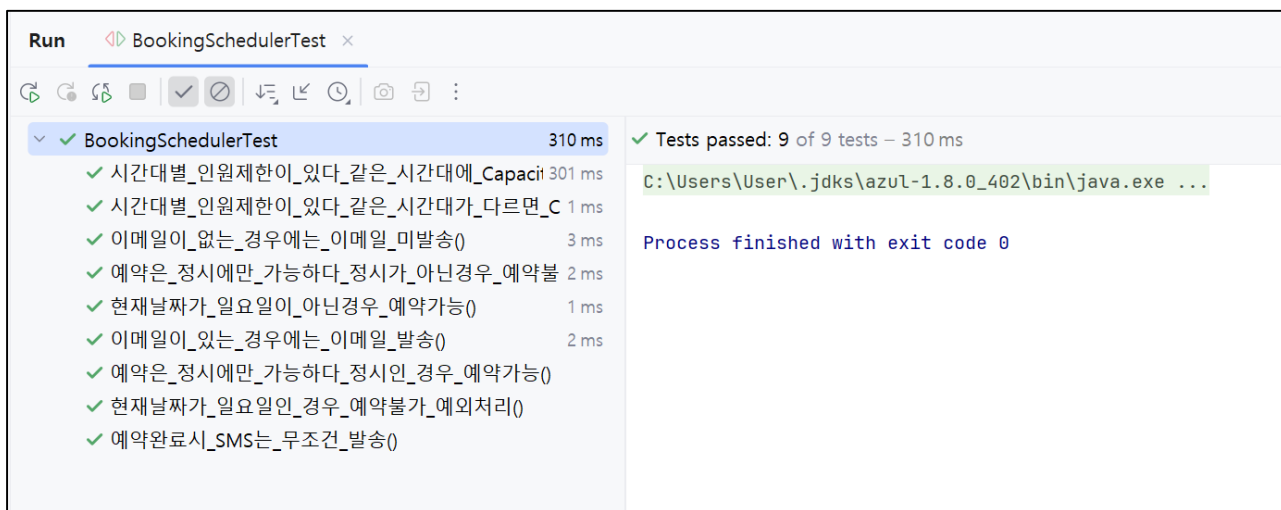
    //act
    bookingScheduler.addSchedule(schedule);

    //assert
    //assertThat(testableMailSender.getCountSendMailMethodIsCalled()).isEqualTo(1);
    verify(mailSender, times(1)).sendMail(schedule);
}
```

TestableSender를 Mock 객체로 대체

✓Step 2-4 : 결과 확인

- TestableMailSender와 TestableSmsSender는 더 이상 필요하지 않으므로 삭제한다.



TestableBookingScheduler를 Mock 객체로 대체

✓Step 3 : TestableBookingScheduler 대신 Mock 객체로 대체

- 현재 PC 날짜를 일요일인 것 처럼 만들어주기 위한 Test Double
- 다음 페이지부터 아래 객체를, Stub 을 사용하는 Spy 객체로 대체할 예정

```
@Test
public void 현재날짜가_일요일인_경우_예약불가_예외처리() {
    //arrange
    BookingScheduler bookingScheduler = new TestableBookingScheduler(CAPACITY_PER_HOUR, dateTime: "2021/03/28 17:00");
    try {
        //act
        Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);
        bookingScheduler.addSchedule(newSchedule);
        fail();
    } catch (RuntimeException e) {
        //assert
        assertThat(e.getMessage(), is(value: "Booking system is not available on sunday"));
    }
}
```

TestableBookingScheduler를 Mock 객체로 대체

✓Step 3-1 : TestableBookingScheduler 대신 Mock 객체로 대체

- bookingScheduler 객체를 @Spy 객체로 만들기

```
@ExtendWith(MockitoExtension.class)
class BookingSchedulerTest {
    public static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm");
    public static final LocalDateTime ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:00", FORMAT);
    public static final LocalDateTime NOT_ON_THE_HOUR = LocalDateTime.parse("2021/03/26 09:05", FORMAT);

    @Mock
    public Customer CUSTOMER;

    @Mock(answer = Answers.RETURNS_MOCKS)
    public Customer CUSTOMER_WITH_MAIL;

    public static final int UNDER_CAPACITY = 1;
    private static final int CAPACITY_PER_HOUR = 3;

    @Spy
    BookingScheduler bookingScheduler;

    @Mock
    public SmsSender smsSender;

    @Mock
    public MailSender mailSender;
```

"@Spy" 적어주기

TestableBookingScheduler를 Mock 객체로 대체

✓Step 3-2 : TestableBookingScheduler 대신 Mock 객체로 대체

- getNow() 메서드 호출시 일요일로 리턴하도록 Stubbing

```
@Test
public void 현재날짜가_일요일인_경우_예약불가_예외처리() {
    //arrange
    LocalDateTime sunday = LocalDateTime.parse("2021/03/28 17:00", FORMAT);
    when(bookingScheduler.getNow()).thenReturn(sunday);
    //BookingScheduler bookingScheduler = new TestableBookingScheduler(CAPACITY_PER_HOUR, "2021/03/28 17:00");

    try {
        //act
        Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);
        bookingScheduler.addSchedule(newSchedule);
        fail();
    }
    catch (RuntimeException e){
        //assert
        assertThat(e.getMessage()).isEqualTo("Booking system is not available on sunday");
    }
}
```

TestableBookingScheduler를 Mock 객체로 대체

✓Step 3-3 : TestableBookingScheduler 대신 Mock 객체로 대체

- `getNow()` 메서드 호출시 월요일로 리턴하도록 Stubbing

```
@Test
public void 현재날짜가_일요일이_아닌경우_예약가능() {
    //arrange
    LocalDateTime monday = LocalDateTime.parse("2024/06/03 17:00", FORMAT);
    when(bookingScheduler.getNow()).thenReturn(monday);
    //BookingScheduler bookingScheduler = new TestableBookingScheduler(CAPACITY_PER_HOUR, "2024/06/03 17:00"); //일요일 아닌 경우

    //act
    Schedule newSchedule = new Schedule(ON_THE_HOUR, UNDER_CAPACITY, CUSTOMER_WITH_MAIL);
    bookingScheduler.addSchedule(newSchedule);

    //assert
    assertThat(bookingScheduler.hasSchedule(newSchedule)).isEqualTo(true);
}
```

TestableBookingScheduler를 Mock 객체로 대체

✓Step 3-4 : 결과 확인



```
Run BookingSchedulerTest x
[Icons]
✓ BookingSchedulerTest 316 ms
  ✓ 시간대별_인원제한이_있다_같은_시간대에_Capacity 302 ms
  ✓ 시간대별_인원제한이_있다_같은_시간대가_다르면_C 1 ms
  ✓ 이메일이_없는_경우에는_이메일_미발송() 4 ms
  ✓ 예약은_정시에만_가능하다_정시가_아닌경우_예약불 2 ms
  ✓ 현재날짜가_일요일이_아닌경우_예약가능() 4 ms
  ✓ 이메일이_있는_경우에는_이메일_발송() 2 ms
  ✓ 예약은_정시에만_가능하다_정시인_경우_예약가능()
  ✓ 현재날짜가_일요일인_경우_예약불가_예외처리() 1 ms
  ✓ 예약완료시_SMS는_무조건_발송()
✓ Tests passed: 9 of 9 tests – 316 ms
C:\Users\User\.jdk\azul-1.8.0_402\bin\java.exe .
Process finished with exit code 0
```